

summarAIse: A Fully-Local AI Lecture-Notes Summariser

1. Project Purpose

summarAIse is a desktop app designed to summarize lecture notes into a short yet accurate summary and a study flashcard set. It's unique because everything is running on the user's own machine: the language model is hosted by Ollama, meaning there is no need for an Internet connection, a paid API, or an account! The tool does not store a user's notes on any cloud service, making it appropriate for private notes, exam preparation and unpublished work which should not be uploaded to a cloud service.

The project is useful because it takes a long time and repetitiveness to turn long notes into study material. summarAIse does this step automatically and allows the control of the output: the user selects the model, uploads one or more files, and gets a Markdown version of the summary which can be downloaded, and an Anki version for active-recall revision. It can import 14 different file formats, and is not limited to plain text, it can read images and text, which means you can use it on real lecture material that might be a mixture of PDFs, slide decks or documents.

Inputs and outputs:

Input: Single file(s) of any of the 14 supported formats (PDF, DOCX, PPTX, XLSX, XLS, TXT, MD, HTML, HTM, CSV, JSON, XML, EPUB, MSG).

Output: summary.md (a Markdown summary) and optionally a flashcards.apkg (Anki flashcard deck).

2. Tools and Libraries Used

It is designed in Python and all the necessary libraries used are open source. Ollama is the local language model. The default model is qwen2.5vl:3b, a small vision capable model. The Python ollama client feeds the summary and passes images of the pages to the model for describing. The web interface is provided by Streamlit: Model picker, Multi-file uploader, Live streaming output, Download buttons.

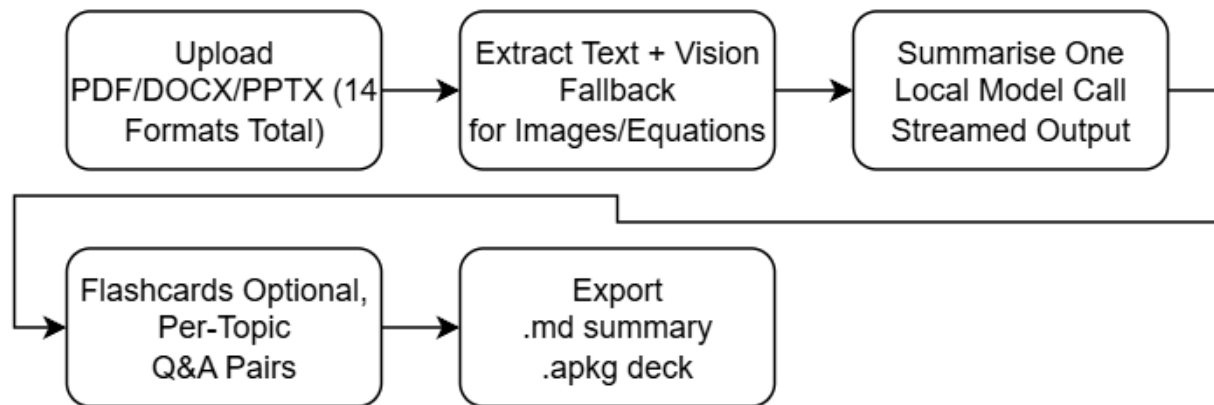
Three libraries are used to extract texts, each optimised for its best formats: PyMuPDF is used for PDFs and supports per-page control and also provides a vision fallback, python-pptx for PowerPoint slides and MarkItDown for all other formats. Both pillow and the vision model are optimized to perform inference quickly, while pillow downscales the page images before they reach vision, and genanki creates the exportable Anki deck. The exact pinned versions can be found on requirements.txt.

3. Workflow

The application is designed around 5 stages of a pipeline. Firstly, one or more files are uploaded by the user. Secondly, every file is being extracted to text. When a page has no text or contains equations which cannot be copied well as text, the page is "rasterised" and its content is described with a vision model, which means that figures and equations are not lost. Thirdly, the whole text of all the files is passed to the chosen model in a single call and it generates the summary. The output is not displayed all at once but only continued as it is produced. Fourthly, the user can optionally turn the summary into flashcards. Fifth, the summary and the flash card deck can be exported. Each stage executes locally on user's machine.

Workflow diagram:

Runs 100% locally via Ollama. No internet, no API keys, no data leaves the machine.



4. Code Explanation

In this section the main part of the code is explained, which contains the function that actually generates the summary. This is found in `summarise.py`, and is shown below, then a simple explanation of it.

`summarise.py`, `summarise_stream()`:

```
def summarise_stream(text, model, depth, proficiency="Intermediate"):
    """Stream a single-pass summary of the whole document in one call."""
    prompt = summary_prompt(text, depth, proficiency)
    client = Client(host=OLLAMA_HOST)
    stream = _with_retry(lambda: client.chat(
        model=model,
        messages=[{"role": "user", "content": prompt}],
        options={"temperature": 0},
        stream=True,
    ))
    for chunk in stream:
        token = chunk["message"]["content"]
        if token:
            yield token
```

This function combines all the text that has been extracted from the uploaded files to generate the summary. It first creates an instruction, using the `summary_prompt()`, which is used to enclose the content in instructions that guide the model in summarising the content. Then it links to the local model via the Ollama client, and transmits the entire document in one call, the one-pass design. The temperature set is 0, so that the model won't give creative output, but will select the safest and most predictable language, keeping the output of the summary as close as possible to the notes. The answer is returned in a stream, so that the user sees the result come

onto the screen gradually, rather than having to wait for the end of the stream. The `_with_retry` helper retries on the failure of the model; a brief glitch will not bring the app down.

5. Test Cases and Results

The application was tested with three representative cases covering its main paths. The figures below are taken from a sample run and should be updated to match your own test.

Case 1, text PDF: a 28-page lecture PDF was uploaded. The application produced a structured Markdown summary in about 4 minutes and 23 seconds on a CPU-only laptop, and the summary was downloaded successfully as `summary.md`. This exercises the fast text-only path.

Case 2, slide deck with images: a PowerPoint file containing equations stored as images was uploaded. The vision model described each figure, and the equations appeared in the resulting summary rather than being skipped, which confirms the vision-fallback path works.

Case 3, flashcards: using the summary from Case 1, the flashcard step generated 17 question-and-answer cards and exported a working `flashcards.apkg` file, which was then imported into the Anki application.

In addition, the helper script `scripts/setup_check.py` was run before launch. It confirmed that the Ollama server was running and that a model was installed, which prevents the most common setup errors.

Sample output 1, generated summary in the app:

Deploy

summarAlse

Lecture notes summariser.

Model

qwen2.5vl:3b

Vision model: reads equations, diagrams, and images in slides/pages.

Upload notes (PDF, DOCX, PPTX, ...)

 Week 14 - up.pdf
374.4KB

+

Summarise

Summary

The lecture focused on leadership and control in open source projects (OSS), emphasizing that OSS leaders are earned through credibility and contributions rather than authority. Key concepts include:

- Leadership vs. Control:**
 - Leadership guides contributors, creates a shared vision, encourages discussion, builds trust, and motivates people to contribute effectively.
 - Control forces decisions, limits discussion, blocks new ideas, creates dependency on one person, and can discourage contributors.
- Corporate vs. OSS Leadership:**
 - Corporate leadership depends on hierarchy and job titles.
 - Open source leadership focuses on trust, openness, and contribution.
 - Titles and positions are less important in OSS communities; real influence comes from meaningful code, documentation, and community contributions.

Sample output 2, generated flashcards:

Deploy

Upload notes (PDF, DOCX, PPTX, ...)

202302...ation.pptx

1.8MB

+

Summarise

Summary

The lecture covered applications of differentiation in calculus, focusing on extreme values and optimization problems. Key concepts included:

- **Absolute and Local Extreme Values:** These are points where a function reaches its maximum or minimum value within a given interval.
- **The Extreme Value Theorem:** States that if a function is continuous on a closed interval $[a, b]$, then it attains both a maximum and a minimum value on that interval.
- **Critical Number:** Points where the derivative of a function equals zero or undefined. These points are potential candidates for local extrema.
- **The Closed Interval Method:** A technique to find absolute extreme values by evaluating the function at critical numbers within an interval and comparing these with boundary values.
- **Example 1 (Mean Value Theorem):** Demonstrates how the Mean Value Theorem can be used to prove that a function satisfies certain conditions over an interval.
- **First Derivative Test:** A method for determining local extrema by examining the sign of the first derivative around critical points. If the sign changes from positive to negative, there is a local maximum; if it changes from negative to positive, there is a local minimum.
- **Concavity:** Indicates whether a curve is concave up or down at any point on its graph.

Additionally, the lecture covered:

Deploy

Flashcards

Generate flashcards

▼

What are the key concepts discussed in the lecture?

Applications of differentiation in calculus, extreme values, optimization problems.

▼

What are absolute and local extreme values?

Points where a function reaches its maximum or minimum value within a given interval.

▼

What is the Extreme Value Theorem?

States that if a function is continuous on a closed interval $[a, b]$, then it attains both a maximum and a minimum value on that interval.

▼

What is a critical number in calculus?

Points where the derivative of a function equals zero or undefined.

▼

What is the Closed Interval Method?

A technique to find absolute extreme values by evaluating the function at critical numbers within an interval and comparing these with boundary values.

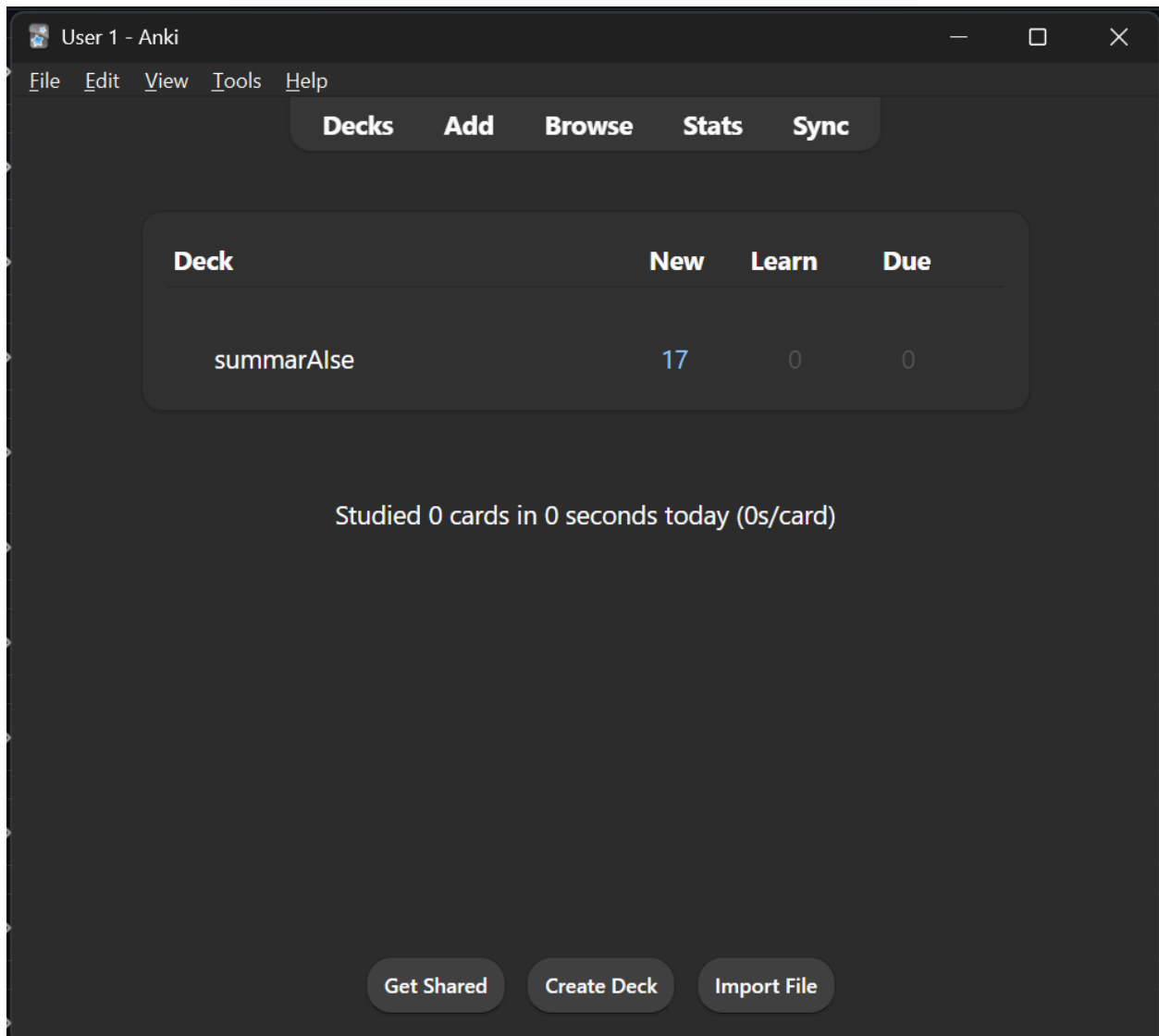
>

What is the Mean Value Theorem?

>

What does the First Derivative Test determine?

Sample output 3, flashcards imported into Anki:



6. Importance of README.md and requirements.txt

The **README.md** is the starting point of a project that is open source. It explains to a new user what the project does, the requirements it has, how to install and run it, the licence and how to overcome common issues. If there is no README, then even if it is correct and well-written, it will be impossible for anyone else except its author to use it, as they don't know how to configure it or what it is used for. A good README will make it possible for other people to adopt and contribute to the project.

This package also contains a **requirements.txt** file that specifies the correct version and external libraries that the project is dependent on. Any user can repeat the same environment set up that the author used with one command: `pip install -r requirements.txt`. This makes the project reproducible: it works the same on another machine, rather than because library A is not installed or it exists in a different version.

In addition to a **LICENSE file** that specifies the legal conditions for reuse, these files allow a folder of code to become a project that others may install, run and build upon.